

RDL TECHNOLOGIES LTD.

Adyar, Mangalore

INTERNSHIP PROJECT REPORT

Embedded Web Server Using STM32F446RCTx & W5500 Ethernet

Serving a Static HTML Web Page over TCP/IP



Figure 1 — Custom STM32F446RCTx + W5500 Board Used in This Project

Microcontroller :	STM32F446RCTx (LQFP64)
Ethernet Module :	WIZnet W5500 (SPI)
IDE :	STM32CubeIDE + CubeMX
Year :	2026

1. Introduction

This report documents the internship project carried out at RDL Technologies Ltd., Adyar, Mangalore. The task was to build a working Embedded Web Server using a custom PCB that has an STM32F446RCTx microcontroller and a WIZnet W5500 Ethernet chip connected over SPI.

The final result is a board that can be accessed from any web browser on the local network. When the user types the board's IP address in the browser, they see an HTML page served directly from the STM32 chip — no external server, no computer required.

1.1 Project Stages

The project was completed in four stages, each verified before moving to the next:

Stage	Name	What Was Done
1	UART Test	Verified that the STM32 can send text to a PC over serial (USB).
2	SPI Test	Confirmed communication with W5500 by reading its version register over SPI.
3	Ping Test	Assigned a static IP to the board and pinged it successfully from a PC.
4	Web Server	Served a complete HTML page from the board to a web browser over HTTP port 80.

1.2 Hardware Used

Component	Model	Purpose
Microcontroller Board	STM32F446RCTx (LQFP64)	Custom PCB — main processing unit
Ethernet Chip	WIZnet W5500	Handles all TCP/IP networking
SPI Bus	SPI2 — PB13, PB14, PB15	Data link between STM32 and W5500
UART Debug	USART3 — 115200 baud	Debug messages from board to PC
Ethernet Cable	RJ45 Cat5e or above	Physical network connection
PC / Laptop	Any Windows or Linux machine	Programming, monitoring, and testing



Figure 2 — The actual custom board used during the internship (W5500 visible at right, STM32 at centre, HanRun RJ45 jack at bottom-right)

2. Software Tools Required

The following software must be installed on the PC before starting. All are free of cost.

2.1 STM32CubeIDE

This is the main IDE used to write, compile, and flash code to the STM32. It includes STM32CubeMX for graphical pin and clock setup.

Download:

<https://www.st.com/en/development-tools/stm32cubeide.html> — Create a free account, download the installer for your OS (Windows or Linux), and run it. Takes about 10–15 minutes to install.

Installation steps:

1. Run the downloaded installer file.
2. Accept the licence agreement.
3. Keep the default installation folder and click Install.
4. Once done, open STM32CubeIDE from the desktop shortcut.

2.2 Serial Terminal (for UART Debug)

A serial terminal shows the debug messages the STM32 sends to the PC. Any one of the following free tools works:

- PuTTY (Windows) — <https://www.putty.org>
- Arduino IDE
- STM32CubeIDE has a built-in serial monitor too

PuTTY Quick Setup:

Open PuTTY → Connection type: Serial → enter COM port (e.g., COM3) → Speed: 115200 → click Open. The correct COM port number can be found in Windows Device Manager under Ports (COM & LPT).

2.3 WIZnet ioLibrary Driver

This is the official C library from WIZnet for the W5500 chip. It provides functions for opening sockets, sending, and receiving data over Ethernet.

Download:

https://github.com/Wiznet/ioLibrary_Driver — Click the green Code button → Download ZIP → Extract it. We only need the files inside the Ethernet folder.

3. Pin Configuration in STM32CubeMX

Pin configuration is done inside STM32CubeIDE, inside .ioc file, which opens automatically when you create a new STM32 project. This step tells the chip exactly which physical pins are used for SPI, UART, reset, and chip select.



Figure 3 — CubeMX Pinout view of STM32F446RCTx showing SPI2, USART3, ETH_RST, and ETH_CS pins assigned

3.1 GPIO Pins

Two GPIO pins are set as digital outputs and controlled manually in code:

User Label	Pin	Default Level	Purpose
ETH_RST	PA6	High	Resets the W5500. Pulled LOW briefly at startup, then HIGH.
ETH_CS	PB12	Low	Chip Select for W5500. Goes LOW to start SPI transfer, HIGH to end.

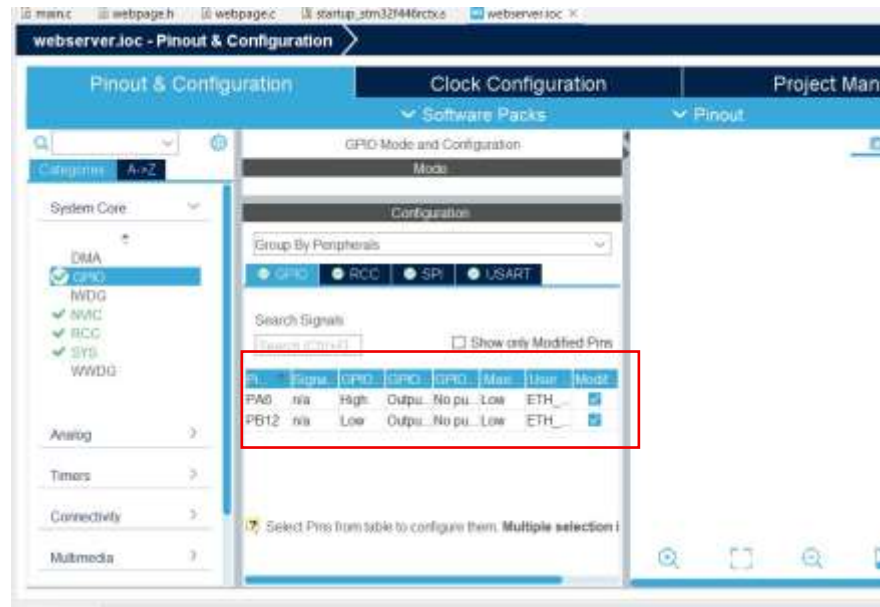


Figure 4 — GPIO configuration in CubeMX showing PA6 (ETH_RST) and PB12 (ETH_CS) as outputs

3.2 SPI2 Configuration

SPI2 is used to communicate between the STM32 and the W5500 chip. The SPI2 physical pins are PB13 (SCK), PB14 (MISO), and PB15 (MOSI).

Parameter	Setting	Reason
Mode	Full-Duplex Master	STM32 drives the clock and controls all timing
Hardware NSS Signal	Disabled	CS is controlled manually in code via ETH_CS pin
Data Size	8 Bits	One byte at a time
First Bit	MSB First	W5500 expects most significant bit first
Clock Polarity (CPOL)	Low	W5500 requires SPI Mode 0
Clock Phase (CPHA)	1 Edge	W5500 requires SPI Mode 0
Baud Rate Prescaler	4 → 11 MBits/s	Well within W5500 max of 80 MHz

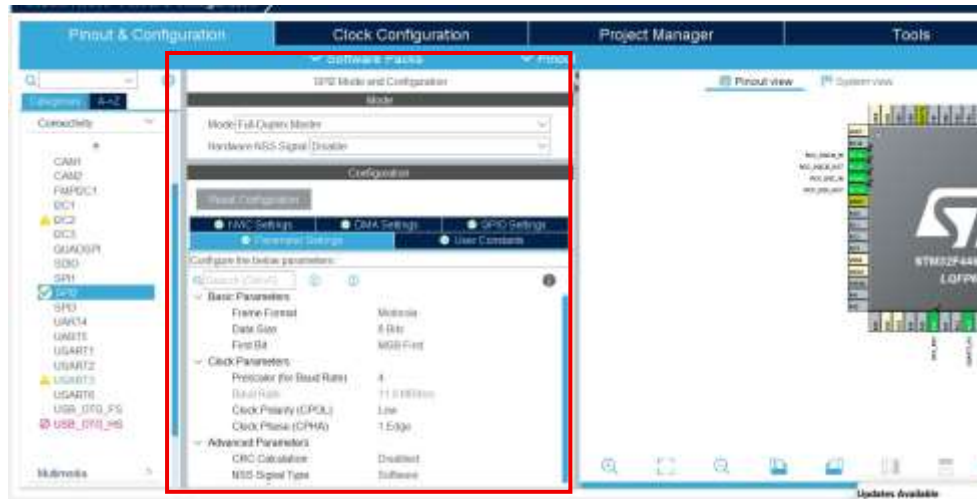


Figure 5 — SPI2 Parameter Settings in CubeMX (Prescaler=4, CPOL=Low, CPHA=1 Edge)

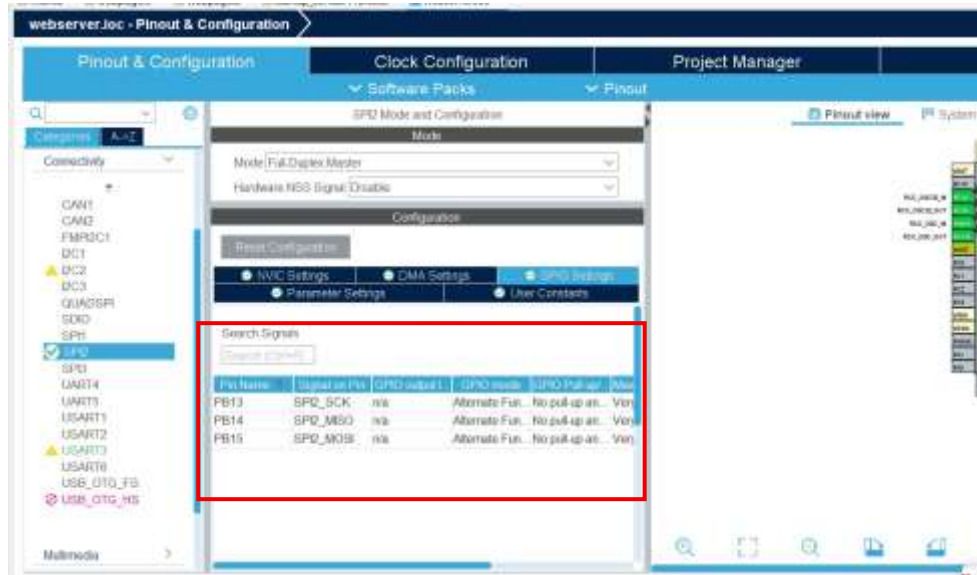


Figure 6 — SPI2 GPIO Settings showing PB13 (SCK), PB14 (MISO), PB15 (MOSI) as Alternate Function

3.3 USART3 Configuration

USART3 is set to Asynchronous mode at 115200 baud. This is how messages like 'W5500 Version = 0x04' and 'Network configured' appear on the PC serial terminal.

Parameter	Setting	Note
Baud Rate	115200	Must match on both STM32 and PC terminal
Word Length	8 Bits	Standard data size
Stop Bits	1	Standard UART framing

Parity	None	No error-check bits
Mode	TX / RX	Both transmit and receive enabled

4. Clock Configuration

The clock configuration sets how fast the STM32 runs. This project uses an external 8 MHz crystal (HSE) and the internal PLL to scale it up to 176 MHz system clock.

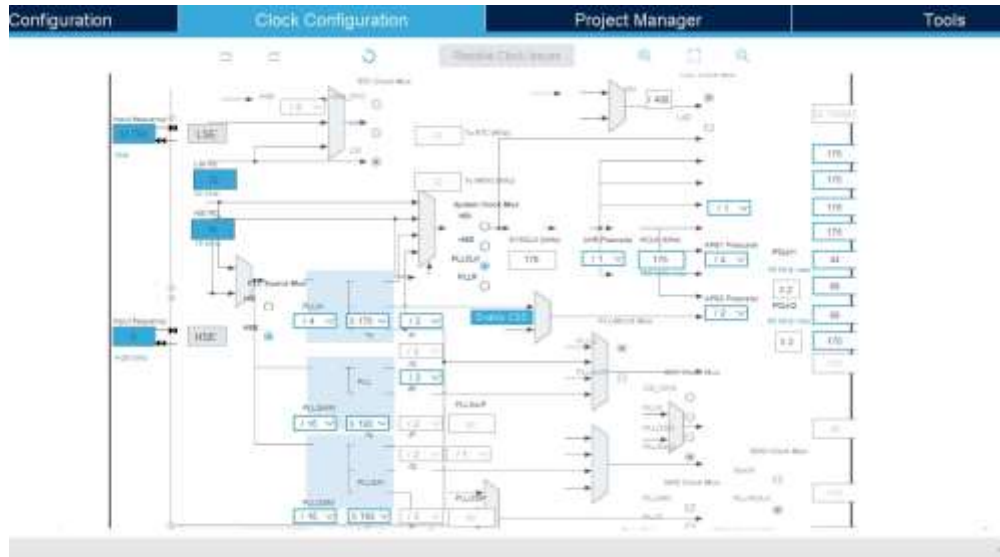


Figure 7 — Clock Configuration tree in CubeMX (HSE 8 MHz → PLL → 176 MHz SYSCLK)

Clock Parameter	Value	Notes
HSE (External Crystal)	8 MHz	Physical crystal on the PCB
PLL Source	HSE	PLL uses the crystal as reference
PLLM Divider	/4	$8 / 4 = 2$ MHz into PLL
PLLN Multiplier	$\times 176$	$2 \times 176 = 352$ MHz VCO
PLLP Divider	/2	$352 / 2 = 176$ MHz SYSCLK
SYSCLK	176 MHz	Main CPU speed
AHB (HCLK)	176 MHz	/1 — core bus
APB1 (PCLK1)	44 MHz	/4 — SPI2 and USART3 clocked from here
APB2 (PCLK2)	88 MHz	/2

Note:

SPI2 peripheral clock = PCLK1 = 44 MHz. With Prescaler = 4, actual SPI clock = $44 / 4 = 11$ MHz. This is safe and well within the W5500 maximum of 80 MHz.

5. Creating the Project in STM32CubeIDE

Follow these steps in order to create the project, configure the pins, add the WIZnet library, and generate the code.

5.1 Create a New STM32 Project

5. Open STM32CubeIDE.
6. Go to File > New > STM32 Project.
7. In the selector window, click the MCU/MPU Selector tab.
8. In the search box, type STM32F446RCT and press Enter.
9. Select STM32F446RCTx from the results and click Next.
10. Enter a project name (e.g., webserver) and click Finish.
11. When asked to open the associated perspective, click Yes. CubeMX will open automatically.

5.2 Apply the CubeMX Configuration

Enable SPI2

12. In the left panel, click Connectivity > SPI2.
13. Set Mode to Full-Duplex Master and Hardware NSS Signal to Disable.
14. In the Parameter Settings tab, set Prescaler to 4. Confirm CPOL = Low, CPHA = 1 Edge.

Enable USART3

15. Click Connectivity > USART3.
16. Set Mode to Asynchronous.
17. Confirm Baud Rate = 115200, Word Length = 8 Bits, Parity = None, Stop Bits = 1.

Configure GPIO Pins

18. On the chip diagram, click pin PA6 and select GPIO_Output.
19. Right-click PA6 > Enter User Label > type ETH_RST.
20. In System Core > GPIO, set PA6 default output level to High.
21. Click pin PB12 and select GPIO_Output.
22. Right-click PB12 > Enter User Label > type ETH_CS.
23. In System Core > GPIO, set PB12 default output level to Low.

Set Clock

24. Click the Clock Configuration tab at the top.
25. Set HSE input frequency to 8 MHz. Select HSE as PLL source.
26. Set PLLM = 4, PLLN = 176, PLLP = /2. Set SYSCLK source to PLLCLK.
27. Set APB1 prescaler to /4 and APB2 prescaler to /2. HCLK should show 176.

Generate Code

28. Click the Project Manager tab at the top of CubeMX.
29. Click Generate Code (or press Alt+K).
30. Click Open Project when prompted.

Important:

CubeMX generates all the initialization code automatically. Never modify the code inside the generated sections. Only write your own code between the USER CODE BEGIN and USER CODE END comment markers.

5.3 Add WIZnet W5500 Library

31. In the Project Explorer panel on the left, right-click Core/Src > New > Folder. Name it W5500.
32. From the extracted ioLibrary ZIP, copy these six files into the W5500 folder:
 - wizchip_conf.h and wizchip_conf.c
 - w5500.h and w5500.c
 - socket.h and socket.c
33. Right-click the project root > Properties > C/C++ General > Paths and Symbols.
34. Add the Core/Src/W5500 folder to the Include Paths list.
35. Click Apply and Close.

5.4 Create webpage.h

36. Right-click Core/Inc folder > New > Header File. Name it webpage.h.
37. Add the following structure inside it:

```
#ifndef WEBPAGE_H
#define WEBPAGE_H

#include <stdint.h>

static const uint8_t index_html[] = {
    /* HTML content as byte array goes here */
}
```

```
    /* See Section 7 for the full explanation */  
};  
  
static const uint32_t index_html_len = sizeof(index_html);  
  
#endif /* WEBPAGE_H */
```

6. Understanding the Code (main.c)

This section explains each important part of the main.c firmware, written in plain language so anyone can follow what is happening and why.

6.1 Header Includes

```
#include "main.h"
#include <stdio.h>
#include "W5500/wizchip_conf.h"
#include "W5500/w5500.h"
#include <string.h>
#include "W5500/socket.h"
#include "webpage.h"
```

These lines tell the compiler which libraries this file needs. The W5500 headers provide all networking functions. webpage.h holds the HTML data. stdio.h enables printf() for debug output.

6.2 printf Redirect to UART

```
int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart3, (uint8_t *)&ch, 1, 100);
    return ch;
}
```

By default, printf() on STM32 does not go anywhere. This function intercepts every character printf() tries to print and sends it over USART3 to the PC serial terminal instead.

6.3 SPI Callback Functions

```
void cs_select(void)    { HAL_GPIO_WritePin(ETH_CS_GPIO_Port, ETH_CS_Pin,
GPIO_PIN_RESET); }
void cs_deselect(void) { HAL_GPIO_WritePin(ETH_CS_GPIO_Port, ETH_CS_Pin,
GPIO_PIN_SET);    }

uint8_t spi_readbyte(void)
{
    uint8_t tx = 0xFF, rx = 0;
    HAL_SPI_TransmitReceive(&hspi2, &tx, &rx, 1, 100);
    return rx;
}

void spi_writebyte(uint8_t wb)
{
    HAL_SPI_Transmit(&hspi2, &wb, 1, 100);
}
```

These four functions teach the WIZnet library how to use our SPI hardware. `cs_select()` pulls `ETH_CS` LOW to start a transfer; `cs_deselect()` pulls it HIGH to end it. `spi_readbyte()` sends a dummy 0xFF and captures what the W5500 sends back — SPI always sends and receives at the same time.

6.4 W5500 Initialisation Sequence

```
// Keep CS HIGH before starting
HAL_GPIO_WritePin(ETH_CS_GPIO_Port, ETH_CS_Pin, GPIO_PIN_SET);

// Hardware reset of W5500
HAL_GPIO_WritePin(ETH_RST_GPIO_Port, ETH_RST_Pin, GPIO_PIN_RESET);
HAL_Delay(10); // Hold LOW for 10 ms
HAL_GPIO_WritePin(ETH_RST_GPIO_Port, ETH_RST_Pin, GPIO_PIN_SET);
HAL_Delay(100); // Wait 100 ms for W5500 to boot

// Register our SPI functions with the library
reg_wizchip_cs_cbfunc(cs_select, cs_deselect);
reg_wizchip_spi_cbfunc(spi_readbyte, spi_writebyte);

// Read version register to confirm SPI works
uint8_t ver = getVERSIONR();
printf("W5500 Version = 0x%02X\r\n", ver); // Should print 0x04
```

The W5500 must be reset at power-on. The reset pin is held LOW for 10 ms, then released. 100 ms is waited afterward to let the chip's internal circuits settle. If `getVERSIONR()` returns 0x04, it confirms SPI is working correctly.

6.5 Network Configuration

```
wiz_NetInfo netinfo = {
    .mac = {0x00, 0x08, 0xdc, 0x11, 0x22, 0x33},
    .ip = {192, 168, 1, 100},
    .sn = {255, 255, 255, 0},
    .gw = {192, 168, 1, 1},
    .dns = {8, 8, 8, 8},
    .dhcp = NETINFO_STATIC
};
wizchip_setnetinfo(&netinfo);
```

Field	Value	Meaning
MAC	00:08:DC:11:22:33	Unique hardware identifier. 00:08:DC is WIZnet's registered prefix.
IP	192.168.1.100	The address you type in a browser to reach the web server.
Subnet	255.255.255.0	All devices 192.168.1.xxx are on the same local network.
Gateway	192.168.1.1	The router's IP — default for most home and office routers.

DHCP	NETINFO_STATIC	We set the IP manually, not assigned by router automatically.
------	----------------	---

Important:

The board IP (192.168.1.100) must be on the same subnet as your PC. Run ipconfig in Command Prompt and check your PC's IP. If your PC shows 192.168.0.xxx, change the board IP to 192.168.0.100.

6.6 Web Server Loop

```
while (1)
{
    switch(getSn_SR(0))          // Check status of socket 0
    {
        case SOCK_CLOSED:
            socket(0, Sn_MR_TCP, 80, 0); // Open TCP socket on port 80
            listen(0);                  // Wait for a browser to connect
            HAL_Delay(1);
            break;

        case SOCK_ESTABLISHED:
            if(getSn_RX_RSR(0) > 0)      // Browser sent a request
            {
                int len = recv(0, rx_buf, sizeof(rx_buf)-1);
                rx_buf[len] = 0;

                if(strncmp((char*)rx_buf, "GET /", 5) == 0)
                {
                    char header[] =
                        "HTTP/1.1 200 OK\r\n"
                        "Content-Type: text/html\r\n"
                        "Connection: close\r\n"
                        "\r\n";
                    send(0, (uint8_t*)header, strlen(header));
                    send(0, (uint8_t*)index_html, index_html_len);
                }
                disconnect(0);
            }
            break;

        case SOCK_CLOSE_WAIT:
            disconnect(0);
            break;
    }
}
```

This is the heart of the web server. It runs in an infinite loop checking the status of Socket 0. The W5500 supports up to 8 sockets. We use socket 0 on TCP port 80 (the standard HTTP port). The behaviour for each state is:

Socket State	Situation	Action Taken
SOCK_CLOSED	No connection open	Open a TCP socket on port 80 and start listening for incoming connections.
SOCK_ESTABLISHED	A browser has connected	Read the HTTP GET request. Send back the HTTP 200 OK header followed by the full HTML page.
SOCK_CLOSE_WAIT	Browser is closing	Cleanly disconnect. Socket returns to CLOSED and the loop repeats.

7. The HTML Web Page

In the initial version of this project shown to the instructor, the HTML was placed directly as a string inside `main.c` and sent as the response body. This is the simplest possible approach and requires no extra files.

7.1 Simple Inline HTML (Initial Version)

```
/* Inside the SOCK_ESTABLISHED case, instead of webpage.h, we used: */

const char html_page[] =
    "<!DOCTYPE html><html><head>"
    "<title>STM32 Web Server</title>"
    "<style>"
    "body { font-family: Arial; text-align: center; background: #f0f0f0; }"
    "h1 { color: #333; margin-top: 60px; }"
    "p { font-size: 16px; }"
    "</style>"
    "</head><body>"
    "<h1>Hello from STM32!</h1>"
    "<p>This page is served by an STM32F446RCTx</p>"
    "<p>using WIZnet W5500 Ethernet.</p>"
    "<p>IP: 192.168.1.100</p>"
    "</body></html>";

send(0, (uint8_t*)html_page, strlen(html_page));
```

When a browser navigates to `http://192.168.1.100`, it sends an HTTP GET request. The STM32 checks if the request starts with 'GET /', and if so, sends back the HTTP header followed by the HTML string above. The browser renders it as a normal web page.

7.2 Upgraded Approach — `webpage.h` Byte Array

For larger HTML pages, storing the code as a C string inside `main.c` becomes messy and hard to edit. The improved approach we moved to afterward converts the HTML file into a C byte array and stores it in a separate file called `webpage.h`. The `main.c` code then just calls:

```
send(0, (uint8_t*)index_html, index_html_len);
```

This keeps `main.c` clean and allows any size of HTML page. The byte array can be generated from an HTML file using a simple Python script or an online hex converter tool.

Note:

This documentation covers the simpler inline HTML version that was the first working version shown to the instructor. The byte-array version is a follow-up improvement done later.

8. Testing Each Stage

We tested the project in four stages. Each stage was verified before moving to the next. This is the right approach in embedded development — always confirm the lower layers work before building on them.

Stage 1 — UART Communication Test

Goal: Confirm that text printed with `printf()` appears on the PC serial terminal.

38. Connect the board to the PC using a USB cable.
39. Open Arduino IDE. Set Connection type: Serial, Speed: 115200, Port: COM3 (or your actual COM port). Click Open.
40. Power on or press the Reset button on the board.

Expected output in the serial terminal:

```
W5500 Version = 0x04
Network configured
Ethernet Cable Connected
```

Troubleshooting:

If nothing appears: (1) Check the correct COM port in Windows Device Manager. (2) Confirm baud rate is exactly 115200. (3) Make sure the USB cable supports data transfer, not just charging.

Stage 2 — SPI Communication Test

Goal: Verify that the STM32 can read data from the W5500 over SPI.

This test is combined with Stage 1. The message 'W5500 Version = 0x04' confirms SPI is working. The value 0x04 is a read-only register inside the W5500. If we can read it correctly, SPI communication is proven.

Serial Output	Meaning	Action
W5500 Version = 0x04	SPI working correctly	Proceed to next stage
W5500 Version = 0x00	CS pin issue	Check PB12 GPIO config in CubeMX
W5500 Version = 0xFF	No SPI response	Check PB13/14/15 pins and W5500 power

Stage 3 — Network Ping Test

Goal: Confirm the board is reachable on the local network from a PC.

41. Connect the board to a router or network switch using an Ethernet cable.
42. Connect the PC to the same router or switch.
43. On Windows, open Command Prompt (Windows key → type cmd → press Enter).
44. Type the command below and press Enter:

```
ping 192.168.1.100
```

Expected output:

```
Pinging 192.168.1.100 with 32 bytes of data:  
Reply from 192.168.1.100: bytes=32 time=1ms TTL=128  
Reply from 192.168.1.100: bytes=32 time=1ms TTL=128  
  
Ping statistics for 192.168.1.100:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

If Ping Fails:

Check: (1) Is the Ethernet cable plugged in? The terminal should show 'Ethernet Cable Connected'. (2) Run ipconfig on the PC — the PC IP should start with 192.168.1.xxx. (3) If another device already uses 192.168.1.100, change the board IP to 192.168.1.101.

Stage 4 — Web Server Test

Goal: Load the HTML page from the STM32 board inside a web browser.

45. Make sure all of Stages 1, 2, and 3 have passed.
46. Open any web browser on the PC (Chrome, Firefox, Edge, etc.).
47. In the address bar, type the following and press Enter:

```
http://192.168.1.100
```

48. Wait 2 to 3 seconds.
49. The HTML web page content should appear in the browser.

Success:

If you see the 'Hello from STM32!' page in the browser, the project is complete and working. The STM32 microcontroller is serving web pages over TCP/IP using the W5500 Ethernet chip.

9. Common Problems and Solutions

Problem	Likely Cause	Solution
Nothing on serial terminal	Wrong COM port or baud rate	Check Device Manager for the correct port. Confirm baud rate is 115200.
W5500 Version = 0xFF	SPI not communicating	Check PB13/14/15 are set as SPI2 in CubeMX. Check ETH_CS is PB12.
'Ethernet Cable NOT Connected'	Physical cable or PHY issue	Try a different Ethernet cable or port on the switch.
Ping fails — Request Timeout	IP address or subnet mismatch	Run ipconfig on PC. Board IP must be in same /24 subnet as PC.
Browser: 'This site can't be reached'	Code not flashed or wrong IP in browser	Confirm the board is running. Use http:// prefix, not https://.
Build error: webpage.h not found	File not in include path	Add Core/Inc to the project include paths in project Properties.
W5500 functions not recognized	WIZnet library not added to project	Confirm W5500 folder is in build path and headers are in include path.

10. Conclusion

This project successfully demonstrated the creation of an Embedded Web Server using a custom STM32F446RCTx microcontroller board and a WIZnet W5500 Ethernet chip, completed as an internship assignment at RDL Technologies Ltd., Adyar, Mangalore.

The project was structured as four progressive milestones — UART communication, SPI communication with the W5500, network connectivity via ping, and finally serving an HTML page from a web browser. All four stages were verified and demonstrated to the instructor.

The firmware uses the WIZnet ioLibrary, standard STM32 HAL drivers, and a simple socket state-machine running on bare metal (no RTOS). The system runs at 176 MHz using the HSE crystal and PLL, with SPI2 communicating to the W5500 at 11 MHz.

Key topics learned through this project include SPI hardware interfacing, TCP/IP networking concepts, HTTP protocol basics, embedded C programming with STM32 HAL, systematic hardware bring-up and debugging, and the use of STM32CubeIDE and CubeMX for rapid development.

References

- STM32F446xx Reference Manual — ST Microelectronics (RM0390)
- W5500 Datasheet — WIZnet Co., Ltd. (v1.0.6)
- WIZnet ioLibrary Driver — github.com/Wiznet/ioLibrary_Driver
- STM32CubeIDE User Guide — ST Microelectronics
- STM32 HAL Driver Documentation — STM32CubeF4 firmware package